

倒排索引

3.1 从逐篇查找到倒排索引

3.1.1 倒排索引的引入

本讲的目标是应用之前学习的数据结构，讨论搜索引擎的设计。我们要解决的第一个问题是，如何高效地返回包含指定词项的文档集合。

假设有下面四篇短文，现在要找同时包含 silver 和 truck 的文档：

文档	内容
D_1	gold silver truck
D_2	shipment of gold damaged in a fire
D_3	delivery of silver arrived in a silver truck
D_4	shipment of gold arrived in a truck

最直接的想法是逐篇扫描，检查是否出现这两个词，得到 D_1, D_3 。然而搜索引擎中的文档数目是巨量的（目前谷歌可索引网页文档数已达千亿级别），并且搜索引擎随时都会面对巨量的查询需求，这样的做法显然不够高效。这就引出了第二个想法：我们可以预先建立词项与文档的索引矩阵，如果一个词项在某篇文档中出现，就在矩阵中对应位置标记为 1，否则标记为 0：

词项	D_1	D_2	D_3	D_4
gold	1	1	0	1
silver	1	0	1	0
truck	1	0	1	1
damaged	0	1	0	0

此时如果要查询同时包含 silver 和 truck 的文档，只需取出 silver 和 truck 两行，按列求逻辑与即可得到 $[1, 0, 1, 0]$ ，也就是 D_1 和 D_3 。然而上表只画出了四个词项。若对真实的搜索引擎建立这一矩阵，设有 V 个不同词项、 N 篇文档，则完整矩阵有 VN 个位置。事实上，读者不难理解，当 V 和 N 都很大时，绝大多数位置取值为 0，因此这会导致巨量的空间浪费。这就促使我们考虑用更精简的方式储存词项与文档的对应关系，也就引入了倒排索引：

定义 3.1. 倒排索引 (inverted index) 由词典与倒排表组成。词典将词项 t 映射到其倒排表 $P(t)$ ；倒排表按文档编号递增，保存含有 t 的文档，必要时还保存词频和位置。

“倒排”的含义是把“文档 $\rightarrow$ 其中的词”的索引反过来，变成“词 $\rightarrow$ 含它的文档”的索引。倒排表按顺序保存文档编号，可以用链表等方式实现。下面给出了上面四篇文档的倒排索引：

编号	词项 (Term)	倒排表 (Times; Documents)
1	a	$\langle 3; 2, 3, 4 \rangle$
2	arrived	$\langle 2; 3, 4 \rangle$
3	damaged	$\langle 1; 2 \rangle$
4	delivery	$\langle 1; 3 \rangle$
5	fire	$\langle 1; 2 \rangle$
6	gold	$\langle 3; 1, 2, 4 \rangle$
7	of	$\langle 3; 2, 3, 4 \rangle$
8	in	$\langle 3; 2, 3, 4 \rangle$
9	shipment	$\langle 2; 2, 4 \rangle$
10	silver	$\langle 2; 1, 3 \rangle$
11	truck	$\langle 3; 1, 3, 4 \rangle$

如上表所示，我们将倒排表写成 $\langle \text{文档数}; \text{文档编号列表} \rangle$ 的形式。例如 silver 对应的 $\langle 2; 1, 3 \rangle$ 表示它出现在两篇文档中，编号分别为 1, 3。分号前的数就是文档频率 $df(t)$ ，它统计含词项 t 的文档数。之后我们还会看到一个概念是词频，即同一个词 t 在一篇文档 d 中出现多少次，用 $tf(t, d)$ 表示。例如 $df(\text{silver}) = 2$, $tf(\text{silver}, D_3) = 2$ 。

3.1.2 用倒排索引回答查询

在理解了倒排索引的结构后，我们可以用它来回答查询。很显然，如果我们要搜索两个词都出现的文档集合，就要求两个倒排表的交集（AND 查询）；如果只要搜索两个词中至少有一个出现的文档集合，就要求两个倒排表的并集（OR 查询）。由于倒排表按文档编号递增排列，因此可以用如下双指针算法高效地求交集和并集。

算法 3.1 有序倒排链表的交集与并集

输入： 编号严格递增的单链表首指针 A, B ；模式 $mode \in \{AND, OR\}$

输出： 编号严格递增且无重复的结果链表的首指针

- 1: $p \leftarrow A$; $q \leftarrow B$; 将 C 的首、尾指针初始化为 $null$
- 2: **while** $p \neq null$ 且 $q \neq null$ **do**
- 3: **if** $p.id = q.id$ **then**
- 4: 向 C 表尾追加 $p.id$; $p \leftarrow p.next$; $q \leftarrow q.next$

```

5:   else if  $p.id < q.id$  then
6:       if  $mode = OR$  then
7:           向  $C$  表尾追加  $p.id$ 
8:       end if
9:        $p \leftarrow p.next$ 
10:  else
11:      if  $mode = OR$  then
12:          向  $C$  表尾追加  $q.id$ 
13:      end if
14:       $q \leftarrow q.next$ 
15:  end if
16: end while
17: if  $mode = OR$  then
18:     while  $p \neq null$  do
19:         向  $C$  表尾追加  $p.id$ ;  $p \leftarrow p.next$ 
20:     end while
21:     while  $q \neq null$  do
22:         向  $C$  表尾追加  $q.id$ ;  $q \leftarrow q.next$ 
23:     end while
24: end if
25: return  $C$  的首指针

```

例如考虑对倒排表 $\langle 3; 1, 2, 4 \rangle$ 和 $\langle 2; 3, 4 \rangle$ 求交集。首先忽略文档频率项，考虑文档链表。先遇到 $1 < 3$ ，因为链表中的数是严格递增的，第二个链表当前以及后面的数都不小于 3，因此 1 不可能再匹配，直接跳过。接着跳过 2，再跳过第二个链表的 3，最后匹配到 4。

有的读者可能会问，文档频率 df 有什么用处。事实上，如果我们要求多个词同时出现的文档集合， df 可以帮助我们选择求交顺序。我们可以先按 df 从小到大排序依次求交，因为求交得到的中间结果不会比参与交集的最短表更长，先处理短表常能较早缩小候选集合，从而减少后续的计算量。

最后，在真实的搜索中，候选文档可能很多，用户难以逐篇阅读。此时，搜索引擎通常会根据评分将结果排序，并按设定的阈值（threshold）只返回前若干篇。

3.1.3 位置索引与短语查询

当我们使用搜索引擎时，搜索引擎在返回结果的同时通常也会节选出包含查询词的上下文片段。为了实现这一功能，我们可以在倒排表中保存每篇文档中词项出现的位置。位置索引还可以帮助我们实现短语查询，例如“silver truck”。如果只知道 silver 和 truck 出现过，而不知道它们在文档中的先后顺序，就无法判断它们是否相邻出现。

下表给出了添加了位置索引的倒排索引。相较于前面的倒排索引，位置索引在每篇文档中保存了词项出现的位置列表。

编号	词项 (Term)	位置倒排表 (Times; Documents Words)
1	a	$\langle 3; (2; 6), (3; 6), (4; 6) \rangle$
2	arrived	$\langle 2; (3; 4), (4; 4) \rangle$
3	damaged	$\langle 1; (2; 4) \rangle$
4	delivery	$\langle 1; (3; 1) \rangle$
5	fire	$\langle 1; (2; 7) \rangle$
6	gold	$\langle 3; (1; 1), (2; 3), (4; 3) \rangle$
7	of	$\langle 3; (2; 2), (3; 2), (4; 2) \rangle$
8	in	$\langle 3; (2; 5), (3; 5), (4; 5) \rangle$
9	shipment	$\langle 2; (2; 1), (4; 1) \rangle$
10	silver	$\langle 2; (1; 2), (3; 3, 7) \rangle$
11	truck	$\langle 3; (1; 3), (3; 8), (4; 7) \rangle$

例如 silver 的 $\langle 2; (1; 2), (3; 3, 7) \rangle$ 表示它出现在两篇文档中：在 D_1 的第 2 个位置出现，在 D_3 的第 3 和第 7 个位置出现。

3.2 索引的构建和维护

3.2.1 文本处理

在理解了倒排索引的结构和查询方法后，我们需要考虑如何建立索引。前面四篇短文的内容很简单，但实际文本可能包含各种标点、连字符、数字等。因此为了建立索引，我们需要先对文本进行处理。

首先我们需要先把文档中的文本拆成词项。前面四篇短文中的单词可以直接用空格分开，但实际文本还会包含标点、连字符和数字，中文也没有用空格标明词的分界，因此需要先确定如何分词。这一步可以由分词器 (tokenizer) 完成。对于英文，通常可以根据空格、标点等识别单词的分界，再按规则处理连字符等情况；中文没有用空格分隔词语，因此需要借助词典或语言模型判断词的分界。本讲不具体展开分词算法，感兴趣的读者可以参考相关资料。

拆分词项时，还有一些细节问题值得讨论。首先考虑是否区分大小写。例如，如果希望 Truck 和 truck 能匹配到相同的文档，就可以统一将字母转为小写。注意如果建立索引时对文档作了这样的处理，查询时也应应对用户输入作相同的处理，才能用处理后的词项查找倒排索引。

同一个词还可能有不同的形式，例如 process、processing 和 processed。如果希望一次查询能找到这些形式，可以进一步进行词干提取 (stemming) 或词形还原 (lemmatization)。词干提取主要依据词形规则去掉词缀，将若干形式归到同一个词干；词形还原则结合词汇知识和语法信息寻找原形，例如把 says、said 还原为 say，这样可以减少因词形不同造成的遗漏。需要注意的是词干提取可能把含义不同的词也归到同一个词干，这样在查询时可能返回不相关的文档。

此外，a、of 等词在大量文档中都会出现，为它们建立倒排表需要较多空间，但它们对查询的区

分作用又很小。因此，一些系统会将这些词列为停用词（stop words），在建立索引时略去。不过，短语查询可能需要这些词，例如“to be or not to be”中的每个词都参与了短语匹配。若要支持这样的查询，就需要保留相应的位置索引。并且，即使决定删除停用词，也应保留其余词在原文中的位置。例如“silver a truck”中 silver 和 truck 的位置是 1, 3；若删除 a 后将它们重新编号为 1, 2，就会错误地认为原文出现了短语“silver truck”。

3.2.2 构建索引

确定词项之后，就可以构建完整词典，进而建立倒排索引。我们需要选择储存词典的数据结构，其中每个词项作为键，储存指向词项对应的倒排表的指针。若共有 V 个词项，使用平衡搜索树查找一个词项需要 $O(\log V)$ 次比较。由于词典是巨量的，因此通常使用 B 树或 B+ 树等数据结构来储存词典。也可以使用哈希表，在散列函数和装载率合适的条件下，查找的期望时间复杂度为常数，但无法支持高效的范围查询。

接下来我们给出构建倒排索引的算法，其大致思路是逐篇读入文档，每读到一个词项，先在词典中找到它的倒排表并添加当前文档编号和位置。若词项不在词典中，就在词典中建立一个新结点。

算法 3.2 在内存中建立位置倒排索引

输入： 按递增编号输入的文档流；确定的文本处理规则

输出： 词典 D 及按文档编号、位置递增排列的倒排表

```

1:  $D \leftarrow$  空词典
2: for 每篇文档  $(d, text)$  do
3:   for 按原位置递增得到的词项与位置  $(t, p)$  do
4:     if  $t$  不在  $D$  中 then
5:       在  $D[t]$  中建立空倒排链表，首、尾指针为  $null$ ,  $df \leftarrow 0$ 
6:     end if
7:     if  $D[t]$  的链表为空，或尾结点的文档编号不等于  $d$  then
8:       新建文档编号为  $d$ 、位置表为空的结点，接到  $D[t]$  的表尾； $df$  加 1
9:     end if
10:    把  $p$  追加到  $D[t]$  尾结点的位置列表
11:  end for
12: end for
13: 每个结点的  $tf$  为位置数；return  $D$ 

```

倒排表的压缩

有一个小的优化可以节省倒排表的存储空间。倒排表中的文档编号递增排列，因此可以保存相邻编号的差值。例如 $[2, 15, 47, 160]$ 对应差值 $[2, 13, 32, 113]$ 。读取差值序列时从 0 开始依次累加，就能恢复全部编号。由于差值通常比原编号小，因此适合采用变长编码，例如按整数大小使用不同数量的字节，让较小的整数占用较少空间，从而节省存储空间。当然这其中存在权衡，因为查询时需要花时间解码。为了解码时能跳过一段较长的倒排表从而加速过程，可以将表分块，并为每块记录起始文档编号和存储位置，使查询能从块边界开始解码。

外存处理

当文档很多时，完整的索引可能放不进内存。因此可以先在内存中建立若干个较小的索引，写入外存后再合并成一个完整索引。因此在算法 3.2 的基础上，还需要加一个判断条件，若内存不足，可以先将当前索引写入外存，再清空内存继续处理后续文档。在处理完所有文档后，再将所有外存索引归并成一个完整索引。

3.2.3 更新与分布式索引

索引建立之后，还需要处理新增、删除和修改的文档。如果每加入一篇文档就重写整个索引，代价会很高，因此可以先为新增文档建立一个较小的辅助索引。查询时同时查找主索引和辅助索引，再合并结果。辅助索引积累到一定大小后，可以与主索引归并。

删除文档时，可以先将它的编号加入删除集合，查询结果中遇到这些编号就将其过滤，等以后归并时再清除对应的倒排表结点。修改文档也可以按类似方式处理：给新版本分配一个编号，将旧版本标为删除。这样，更新操作可以先完成较小的修改，而将整理整个索引的工作推迟到归并阶段。

当全部文档的索引太大，一台机器难以保存或处理时，可以让多台机器分别负责其中的一部分，也就是分布式索引。划分时有两种思路：按词项分，或者按文档分。下面仍用前面的四篇短文说明。

按词项划分 (term-partitioned index)，就是将词典中的词项分给不同机器，每台机器保存所负责词项的完整倒排表。例如，可以按字母顺序分配词项，让第一台机器负责 a-s 开头的词项，第二台负责 t-z 开头的词项。这样，silver 的倒排表 $\langle 2; 1, 3 \rangle$ 保存在第一台，truck 的倒排表 $\langle 3; 1, 3, 4 \rangle$ 保存在第二台。

查询 silver 时，只需访问第一台机器；查询同时包含 silver 和 truck 的文档时，则需要从两台机器取得相应的文档编号并求交，得到 D_1, D_3 。也可以先取得其中一个倒排表，将其中的文档编号发给另一台机器筛选，因此多词查询可能需要在机器之间传输倒排表或中间结果。

按文档划分 (document-partitioned index)，就是先将文档分给不同机器，再由每台机器为自己负责的文档建立索引。例如，第一台负责 D_1, D_2 ，第二台负责 D_3, D_4 。两台机器都需要保存 silver 和 truck 的倒排表，但第一台的表只记录这两个词在 D_1, D_2 中的出现情况，第二台的表只记录它们在 D_3, D_4 中的出现情况。

查询同时包含 silver 和 truck 的文档时，将同一个查询发给两台机器。第一台在本地求交，找到 D_1 ；第二台在本地求交，找到 D_3 。最后将两台机器返回的结果汇总，便得到 D_1, D_3 。由于每台机器负责的文档不同，这些结果可以直接合并，机器之间无需交换完整倒排表。

按词项划分时，单词查询只需访问负责该词的机器，但热门词可能使某台机器承担大量查询。按文档划分时，多词求交可以在各台机器内部完成，不过为了找齐全部匹配文档，通常要向所有机器发送查询。因此，实际选择时需要结合查询的词数、热门词的分布，以及机器之间的通信开销等。

3.3 搜索结果的评价

3.3.1 数据检索

在理解了倒排索引的建立和查询方法后，我们需要考虑如何评价搜索结果。首先介绍数据检索的评价方法。数据检索 (data retrieval) 是按照明确、可判定的查询条件，从给定的数据集合中找出满足条件的数据。例如，查询 silver AND truck 时，只要检查文档是否同时包含两个词，就能判断它

是否应当出现在结果中。

对于数据检索，首先要确认结果正确，然后在评价检索系统的处理能力时，还可以具体考察以下几个方面：

- **建立索引的速度**。例如，每小时能够为多少篇文档建立索引。这个指标反映了系统处理新增文档的能力。
- **查询的速度**。从提交查询到返回结果需要多长时间，这也被称为查询延迟 (latency)。除了测量给定规模下的延迟，还应观察索引增大时，查询延迟如何变化。
- **查询语言的表达能力**。用户能否准确表达较复杂的需求，例如“同时包含 silver 和 truck，但不包含 damaged”，或者要求几个词连续出现。系统既要支持这些查询，也要考察执行复杂查询所需的时间。

3.3.2 信息检索

用户使用搜索引擎时，通常希望找到能回答自己问题的内容。例如，输入 silver 和 truck，可能是想了解一批白银的运输情况，但仅仅找到同时包含这两个词的文档，还不能保证文档回答了这个问题。这就引出了一个和数据检索不同的概念：信息检索 (information retrieval)。信息检索是根据用户的信息需求，从文档等信息资源中查找与该需求相关的内容。用户通常通过查询表达需求，系统据此判断文档的相关程度，并组织检索结果；结果的质量取决于它们能否满足用户的需求。

查询词只能表达用户需求的一部分，因此可能出现两种情况：一篇文档包含了全部查询词，但讨论的主题与用户的问题无关；另一篇文档能够回答问题，却采用了不同的说法，没有出现这些查询词。通常而言，评价信息检索效果时，我们既要查看返回的文档中有多少是用户需要的，也要看需要的文档有多少没有被找到，下面用两个指标分别描述这两点。设用于评测的文档集合为 $\mathcal{U}$ 。对于一个给定查询，先根据相关性标注相关文档集合 $\mathcal{R}$ ，再将系统返回的文档集合记为 $\mathcal{S}$ 。按是否相关、是否返回，可以将 $\mathcal{U}$ 中的文档分为下面四类：

	相关	不相关
返回	$TP = \mathcal{S} \cap \mathcal{R} $	$FP = \mathcal{S} \setminus \mathcal{R} $
未返回	$FN = \mathcal{R} \setminus \mathcal{S} $	$TN = \mathcal{U} \setminus (\mathcal{R} \cup \mathcal{S}) $

注意其中 T 是“True”的首字母，F 是“False”的首字母，P 是“Positive”的首字母，N 是“Negative”的首字母。TP、FP、FN、TN 分别表示真正例、假正例、假反例和真反例的数量，或者也称真阳性、假阳性、假阴性和真阴性。

定义 3.2. 查准率或精确率 (precision) 衡量返回的文档中有多少相关，**查全率或召回率 (recall)** 衡量相关文档有多少被找到：

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}.$$

若系统没有返回任何文档，精确率的分母为零；若评测集合中没有相关文档，召回率的分母为零。我们将这两种情况下的相应指标记为“未定义”。

精确率只考察已经返回的文档。如果将返回相关文档和未返回不相关文档都算作判断正确，就得到准确率 (accuracy) $(TP + TN)/|U|$ 。在检索任务中，不相关文档可能占绝大多数。例如一千篇文档中只有十篇相关，系统即使什么都不返回，准确率仍有 99%，但十篇相关文档全部漏掉了。因此，精确率和召回率能更直接地反映搜索结果是否有用。

例题 3.3. 评测集合共 100 篇，相关文档 20 篇，系统返回 15 篇，其中 12 篇相关。则 $TP = 12, FP = 3, FN = 8, TN = 77$ ，精确率为 $12/15 = 0.8$ ，召回率为 $12/20 = 0.6$ 。

如果系统将文档按评分从高到低排列，并返回前 k 篇，那么增大 k 后，原来已经找到的相关文档仍在结果中，召回率因而不会降低。精确率的整体变化趋势则是降低，因为好的评分规则下，前面返回的文档更可能相关，而后面返回的文档可能不相关。但我们也可以构造例外情况，例如前三篇文档依次为“相关、不相关、相关”，返回前两篇时精确率为 $1/2$ ，返回前三篇时反而提高到 $2/3$ 。但整体而言，精确率和召回率是相互制约的，在实际应用中往往需要在两者之间作权衡。

有时还希望用一个数综合精确率和召回率，这就引出了它们的调和平均数，称为 F_1 分数：

$$F_1 = \frac{2PR}{P + R}.$$

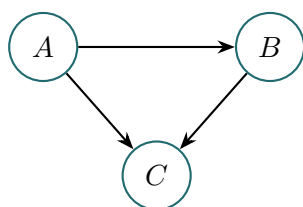
当这些分式有定义时，代入 P 和 R 可以得到 $F_1 = 2TP/(2TP + FP + FN)$ 。值得注意的是， F_1 分数的调和平均数定义意味着，若精确率和召回率相差很大， F_1 分数会偏向较小的那个指标，因此在实际应用中，如果要达到较高的 F_1 分数，则需要同时保证精确率和召回率都不低。

3.4 补充材料：网页评分算法

3.4.1 基本思路

使用倒排索引找到包含查询词的网页后，搜索引擎还需要决定这些网页的展示顺序。一个常用的思路是考察网页之间的链接：如果许多网页都链接到某篇文章，这篇文章可能值得优先阅读；如果推荐它的网页本身也很有影响力，这种推荐可能更有价值。接下来要介绍的 HITS 和 PageRank 都利用了这样的相互影响，为网页计算分数。

用有向图 $G = (V, E)$ 表示选定的有限网页集合，记网页数为 $n = |V|$ 、链接数为 $m = |E|$ 。边 $i \rightarrow j$ 表示网页 i 链接到网页 j 。在下面的分析中，网页集合和链接保持不变，网页按固定顺序编号。此外，同一对网页之间的重复链接只计一次，且网页指向自己的链接略去。例如图 3.1 中， A 链接到 B, C ， B 链接到 C ，而 C 没有指向其他网页的链接。



C 没有出链

图 3.1 用于演示 HITS 和 PageRank 的链接图。箭头表示网页之间已有的链接。

3.4.2 HITS 算法

确定与查询有关的网页集合

一篇专业文章的价值可能在于它提供的内容，一个资料目录的价值则可能在于它收集了许多值得阅读的文章。HITS (Hyperlink-Induced Topic Search) 分别用 authority 和 hub 两种分数评价网页在这两方面的作用。其中，authority score a_i 根据哪些网页链接到网页 i ，评价它作为内容来源的权威程度；hub score h_i 根据网页 i 链接到哪些网页，评价它推荐内容的能力。一个网页可以同时提供内容和推荐其他网页，所以算法会为每个网页计算这两种分数。

HITS 通常先根据当前查询选出一批网页，再对这些网页评分。首先通过文本检索得到一批与查询有关的网页，称为根集合 R_0 。接着，从两个方向扩展这个集合：既加入这些网页推荐的网页，也加入推荐了这些网页的网页。扩展后得到的集合称为基础集合 B_0 。以 B_0 为顶点集合，保留两端都在其中的链接，就得到用于评分的图。需要注意的是，如果要查找某个网页推荐了谁，可以直接读取它的出链；而如果要查找谁推荐了它，则需要事先建立反向链接索引。

为了控制规模并简化讨论，本讲采用如下基础集合的取法：根集合至多包含 r 个网页，对其中每个网页，至多加入 b 个链接到它的网页和 b 个它链接到的网页；超过限额时，按网页编号从小到大选择。这样得到的基础集合至多包含 $r(1 + 2b)$ 个顶点。

迭代公式

在 HITS 中，一个网页得到高 hub 网页的推荐，会增加它的 authority；它推荐了高 authority 的网页，又会增加它自己的 hub。下面的迭代公式体现了这种相互影响：

$$a'_j = \sum_{i:i \rightarrow j} h_i, \quad h'_i = \sum_{j:i \rightarrow j} a'_j.$$

计算从一组给定的 hub 初值开始，通常让所有网页的初值相同。先用第一个式子计算各网页的 authority，再将刚算出的 authority 代入第二个式子更新 hub，完成一轮迭代。初值相同时，每条入链在第一轮提供相同的贡献，所以第一次得到的 authority 与入度成比例。下一轮使用更新后的 hub，各条入链的贡献就会因来源网页不同而有所区别。

反复求和可能使分数越来越大，而网页排序只取决于分数的相对大小。因此，每次更新 authority 后，将整个 authority 向量除以它的长度；更新 hub 后，也对 hub 向量作同样处理，使这两个向量的长度都保持为 1。这里使用欧几里得范数 $\|x\|_2 = \sqrt{\sum_i x_i^2}$ 。这样的归一化不会改变分数的大小次序。为了用矩阵表示计算过程，定义邻接矩阵 A ：存在边 $i \rightarrow j$ 时 $A_{ij} = 1$ ，否则 $A_{ij} = 0$ ，分数向量均写成列向量。于是第 t 轮的迭代公式为

$$a^{(t)} = \frac{A^T h^{(t-1)}}{\|A^T h^{(t-1)}\|_2}, \quad h^{(t)} = \frac{A a^{(t)}}{\|A a^{(t)}\|_2}.$$

其中 A^T 是 A 的转置。观察矩阵乘法的第 j 个分量， $(A^T h)_j = \sum_i A_{ij} h_i$ ，恰好是在累加所有链接到 j 的网页的 hub 分数；同理， $(A a)_i = \sum_j A_{ij} a_j$ 累加的是网页 i 所推荐的各网页的 authority 分数。只要图中至少有一条边，且 hub 初值的各分量都为正，第一次计算的 $A^T h$ 就非零，可以归一化。设某一轮得到 $a = A^T h / \|A^T h\|_2$ ，则 $h^T A a = (A^T h)^T a = \|A^T h\|_2 > 0$ ，所以 $A a$ 也非零。更新为 $h' = A a / \|A a\|_2$ 后，又有 $a^T A^T h' = \|A a\|_2 > 0$ ，下一轮的 $A^T h'$ 因而仍非零。这样就保证了以后各轮的归一化都能进行。若图中没有边，所有贡献都为零，算法将返回全零分数，并说明无法根据链接区分网页。

算法 3.3 HITS 的迭代算法

输入： 固定图 $G = (V, E)$, $n \geq 1$, 迭代次数 $K \geq 1$

输出： K 轮的 authority、hub 近似分数；分数相同按网页编号排序

```

1: if  $E = \emptyset$  then
2:   return 全零分数, 并报告“没有链接, 无法进行排名”
3: end if
4:  $h \leftarrow (1/\sqrt{n}, \dots, 1/\sqrt{n})$ 
5: for  $t = 1, \dots, K$  do
6:    $a \leftarrow \mathbf{0}$ 
7:   for 每条边  $i \rightarrow j$  do
8:      $a_j \leftarrow a_j + h_i$ 
9:   end for
10:   $a \leftarrow a/\|a\|_2$ 
11:   $g \leftarrow \mathbf{0}$ 
12:  for 每条边  $i \rightarrow j$  do
13:     $g_i \leftarrow g_i + a_j$ 
14:  end for
15:   $h \leftarrow g/\|g\|_2$ 
16: end for
17: return  $(a, h)$ 

```

不难计算 HITS 的时间复杂度：每轮更新遍历两次边，归一化时再遍历向量的 n 个分量，因此每轮时间为 $O(n + m)$ ，故 K 轮共需 $O(K(n + m))$ 时间。当然实际检索还需要加上构建基础集合的时间。此外，若最后要将全部网页按分数排序，还需 $O(n \log n)$ 时间。

计算示例

图 3.1 的邻接矩阵为

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

设三个网页的初始 hub 相同。 C 有两条入链， B 有一条，而 A 没有入链，因此第一次得到的 authority 之比为 $0 : 1 : 2$ 。归一化后， B, C 的 authority 分别为 $1/\sqrt{5}, 2/\sqrt{5}$ 。接着更新 hub： A 推荐了 B, C ，得到两者的分数之和 $3/\sqrt{5}$ ； B 只推荐了 C ，得到 $2/\sqrt{5}$ ； C 没有出链，得到 0。将 hub 归一化，就有

$$a^{(1)} = \frac{(0, 1, 2)^T}{\sqrt{5}}, \quad h^{(1)} = \frac{(3, 2, 0)^T}{\sqrt{13}}.$$

第二轮同理得到

$$a^{(2)} = \frac{(0, 3, 5)^T}{\sqrt{34}}, \quad h^{(2)} = \frac{(8, 5, 0)^T}{\sqrt{89}}.$$

读者可以继续迭代，观察分数的变化。并且不难证明从相同的正 hub 初值出发，每轮迭代得到的分数中， C 的 authority 最高。

迭代的收敛性

一个自然的问题是，上述迭代是否会收敛到一个稳定的分数向量。我们可以做如下分析：每轮迭代先计算 $A^T h$ ，再左乘 A ，两次归一化只改变向量的长度。因此，做完 t 轮后， $h^{(t)}$ 的方向与 $Q^t h^{(0)}$ 相同，其中 $Q = AA^T$ 。这种反复乘以同一个矩阵、再归一化的过程称为幂迭代（power iteration）。要判断向量的方向最终是否稳定，可以将初值分解到矩阵的各个特征方向上，分别考察它们在迭代中的变化。

矩阵 $Q = AA^T$ 是实对称矩阵，并且对任意向量 x 都有 $x^T Q x = \|A^T x\|_2^2 \geq 0$ ，因此它的特征值都非负。根据实对称矩阵的谱定理，可以选择单位正交的特征向量 $u_1, \dots, u_n$ ，作为空间的一组基。将对应特征值按大小排列，不妨令 $\lambda_1 \geq \dots \geq \lambda_n \geq 0$ 。由于 $Q u_i = \lambda_i u_i$ ，每将向量左乘一次 Q ，它在 u_i 方向上的系数就乘上 λ_i 。如果初值在最大特征值对应方向上的线性表示系数非零，且这个特征值严格大于其余特征值，那么其余方向的系数相对于它都会逐渐趋于零，归一化后的向量也就趋向这个特征方向。下面的定理形式化了这个结论，并说明了极限的唯一性。

定理 3.4. 若图至少有一条边，且 $Q = AA^T$ 的最大特征值是单重的，即 $\lambda_1 > \lambda_2$ ，则从全正 hub 初值出发，归一化 HITS 使得 hub 向量和 authority 向量分别收敛到唯一的非负单位向量。

证明. 首先要保证初始向量在最大特征方向上的线性表示系数不为零。对任意单位向量 $u = \sum_i b_i u_i$ ，有 $u^T Q u = \sum_i \lambda_i b_i^2 \leq \lambda_1 \sum_i b_i^2 = \lambda_1$ 。取一个对应 λ_1 的单位特征向量，将各分量换成绝对值，得到向量 w 。由于 Q 的所有元素非负，换成绝对值后 $w^T Q w$ 不会减小；另一方面， w 仍是单位向量，所以上式又保证 $w^T Q w \leq \lambda_1$ ，于是只能取等号，因此 w 仍是对应 λ_1 的单位特征向量。这样便可以选取各分量非负的 u_1 。它至少有一个正分量，而 $h^{(0)}$ 的每个分量都为正，所以内积 $c_1 = (h^{(0)})^T u_1 > 0$ 。从而可以将初值展开为 $h^{(0)} = \sum_i c_i u_i$ ，乘 Q^t 再除以 λ_1^t 得到

$$\frac{Q^t h^{(0)}}{\lambda_1^t} = c_1 u_1 + \sum_{i=2}^n c_i \left(\frac{\lambda_i}{\lambda_1} \right)^t u_i.$$

图中至少有一条边，因而 Q 至少有一个正特征值， $\lambda_1 > 0$ 。对 $i \geq 2$ ，有 $0 \leq \lambda_i/\lambda_1 < 1$ ，所以上式右侧除第一项外都趋于零，整个向量趋于 $c_1 u_1$ 。再除以向量长度，就得到 $h^{(t)} \rightarrow u_1$ 。由于要求 u_1 非负且长度为 1，所以极限 $h^* = u_1$ 是唯一的。

接着考察 authority，只需对 authority 的迭代公式取极限即可得到 $a^* = A^T u_1 / \sqrt{\lambda_1}$ 。这个向量非负（注意 A 和 u_1 中的元素都是非负的），长度为 1，根据 h^* 的唯一性， a^* 也随之唯一确定。□

回顾图 3.1 的例子，

$$Q = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad \lambda_{1,2} = \frac{3 \pm \sqrt{5}}{2}, \quad \lambda_3 = 0.$$

令 $\varphi = (1 + \sqrt{5})/2$ ，解 $Qh = \lambda_1 h$ 得 $h^* = \frac{(\varphi, 1, 0)^T}{\sqrt{\varphi^2 + 1}}$ ， $a^* = \frac{(0, 1, \varphi)^T}{\sqrt{\varphi^2 + 1}}$ 。

上面的定理表明，任意全正初值最后都能迭代到同一个极限向量。但如果最大特征值对应多个独立的特征方向，不同的初值可能会收敛到不同的极限向量：本质而言，极限向量是初值在最大特征

值对应的特征子空间上的归一化投影，如果最大特征值重数为 1，则对应的特征子空间是一维的，投影方向唯一；否则不同的初值可能在这个子空间上有不同的投影方向，迭代结果也就不同。

实际应用中，HITS 的分数还受基础集合的影响。围绕查询扩展网页集合时，可能加入一批链接密集但主题与查询无关的网页。这些网页间的链接会在迭代中反复贡献分数，使它们可能排在真正相关的网页之前。人为增加互相推荐的链接也会产生类似影响。因此，使用 HITS 排序时，需要检查选入的网页是否仍围绕查询主题，以及高分是否主要来自一批无关网页间的大量相互链接。

3.4.3 PageRank 算法

基本思想

与 HITS 算法不同，PageRank 根据浏览网页的过程定义分数。假设用户当前位于网页 i ，它有 d_i 条出链，用户等概率选择一条继续浏览，那么每条出链被选中的概率就是 $1/d_i$ 。如果一个网页经常被访问，它所链接的网页也就有更多被访问的机会。PageRank 用这个随机浏览过程达到稳定后，用户位于各网页的概率作为评分。要这样定义分数，首先需要保证位置分布确实能趋于稳定。

不过，仅沿链接浏览会遇到几个问题。例如，如果图中只有 $A \rightarrow B, B \rightarrow A$ ，从 A 开始时，所在网页会一直在 A, B 之间交替，对应的概率分布也在 $(1, 0)^T$ 和 $(0, 1)^T$ 之间交替，无法趋于稳定。此外，如果图中有两个互不连通的部分，用户始终留在出发时所在的部分，长期访问情况就会受到起点的限制。

为了解决这些问题，在沿链接浏览之外，再允许用户随机跳转到一个网页。用概率向量 v 表示这次选择，满足 $v_i \geq 0$ 且 $\sum_i v_i = 1$ ，其中 v_i 是选择网页 i 的概率。最简单的选择是均匀分布，即 $v_i = 1/n$ 。给定 $0 < \alpha < 1$ ，用户在有出链的网页上，以概率 α 沿一条随机选中的出链浏览，以概率 $1 - \alpha$ 按分布 v 跳转。在没有出链的网页上，则直接按 v 跳转。

转移矩阵与迭代公式

用列向量 $p^{(t)}$ 表示第 t 步的位置分布，其中 $p_j^{(t)}$ 是用户位于网页 j 的概率。根据上一小节的规则，定义如下矩阵 P ：

$$P_{ij} = \begin{cases} 1/d_j, & d_j > 0 \text{ 且 } j \rightarrow i, \\ 0, & d_j > 0 \text{ 且无 } j \rightarrow i, \\ v_i, & d_j = 0. \end{cases}$$

其中 P_{ij} 表示从网页 j 到达网页 i 的概率，所以出发网页对应列，到达网页对应行。有出链时，网页 j 的 d_j 个链接目标各得到概率 $1/d_j$ ；没有出链时，各网页按 v 得到概率。因此， P 的元素非负，每列之和为 1，这样的矩阵称为列随机矩阵。没有出链的网页称为悬挂结点（dangling node），它对应的整列就是 v 。进一步地，再加入概率为 $1 - \alpha$ 的随机跳转，就可以得到完整的迭代公式

$$p^{(t+1)} = \alpha P p^{(t)} + (1 - \alpha)v. \quad (3.1)$$

定义 3.5. 定义 PageRank 向量 p^* 是满足

$$p^* = \alpha P p^* + (1 - \alpha)v, \quad p_i^* \geq 0, \quad \sum_i p_i^* = 1 \quad (3.2)$$

的稳定分布。这里的稳定指：如果当前分布是 p^* ，再走一步仍为 p^* 。

接下来证明这个方程有唯一解，并且从任意概率分布开始迭代都能收敛到该解。

收敛性分析

对向量 x 定义 $\|x\|_1 = \sum_i |x_i|$ 。因为 P 非负且每列之和为 1，

$$\|Px\|_1 = \sum_i \left| \sum_j P_{ij} x_j \right| \leq \sum_{i,j} P_{ij} |x_j| = \sum_j |x_j| = \|x\|_1.$$

因此，左乘 P 后，向量的 L_1 范数不会增大。将一次迭代记为 $F(p) = \alpha Pp + (1 - \alpha)v$ ，对两个概率向量 p, q 应用上式，便有

$$\|F(p) - F(q)\|_1 \leq \alpha \|p - q\|_1. \quad (3.3)$$

也就是说，对两个分布各做一次迭代，所得分布的距离至多为原距离的 α 倍。由于 $\alpha < 1$ ，重复迭代会不断缩小这种距离。

定理 3.6. 对任意概率向量 v 和任意 $0 < \alpha < 1$ ，定义 3.5 中的 PageRank 向量 p^* 存在且唯一。从任意概率初值 $p^{(0)}$ 出发，按式 (3.1) 迭代得到的向量 $p^{(t)}$ 收敛到 p^* ，且

$$\|p^{(t)} - p^*\|_1 \leq 2\alpha^t.$$

证明. 由于 P 是列随机矩阵，且 v 是概率向量， $F(p)$ 的分量仍非负、和仍为 1，所以每轮得到的都是概率向量。记相邻两轮的距离为 $\Delta_t = \|p^{(t)} - p^{(t-1)}\|_1$ ，由式 (3.3) 可得 $\Delta_{t+1} \leq \alpha \Delta_t$ 。因此，对任意 $s > t$ ，

$$\|p^{(s)} - p^{(t)}\|_1 \leq \sum_{j=t+1}^s \Delta_j \leq \frac{\alpha^t}{1 - \alpha} \Delta_1 \rightarrow 0.$$

这个上界与 s 无关，并且随 t 增大趋于零，所以各坐标都构成实数中的柯西序列，因而分别收敛。由于坐标只有有限个，故极限向量仍然非负、分量之和仍为 1。对式 (3.1) 取极限，就得到满足 $p^* = F(p^*)$ 的概率向量。

再证明唯一性。若另有一个极限向量 q^* ，则

$$\|p^* - q^*\|_1 = \|F(p^*) - F(q^*)\|_1 \leq \alpha \|p^* - q^*\|_1,$$

因为 $0 < \alpha < 1$ ，所以只能有 $\|p^* - q^*\|_1 = 0$ ，即 $p^* = q^*$ 。

最后将 $p^{(t)}$ 与 p^* 比较。反复使用式 (3.3)，并注意到两个概率向量的 L_1 距离至多为 2，可得

$$\|p^{(t)} - p^*\|_1 \leq \alpha^t \|p^{(0)} - p^*\|_1 \leq 2\alpha^t.$$

□

上述证明只使用了 P 是列随机矩阵， $0 < \alpha < 1$ ，以及 v 是概率向量。因此无论原链接图是否连通、是否含有悬挂结点，也无论选取哪个概率向量 v ，唯一性和收敛性都成立。如果 v 的每个分量都为正，还能由式 (3.2) 得到 $p_i^* \geq (1 - \alpha)v_i > 0$ ，即每个网页的分数都为正。若 v 中有零分量，某些网页的分数可能为零。

实际计算时，我们还不知道 p^* ，但可以算出相邻两轮的距离 Δ_t 。由 $\Delta_{t+1} \leq \alpha \Delta_t$ 可知，后续各轮的变化依次不超过 $\alpha \Delta_t, \alpha^2 \Delta_t, \dots$ 。将这些变化相加，就得到当前结果到极限的误差界为 $\|p^{(t)} - p^*\|_1 \leq \frac{\alpha}{1 - \alpha} \Delta_t$ 。因此，只要 $\Delta_t \leq \varepsilon(1 - \alpha)/\alpha$ ，当前结果与 p^* 的 L_1 距离就不超过 ε ，这就给出了一个可以直接检查的停止条件。

计算示例

仍考虑图 3.1, 取均匀跳转分布 $v = (1/3, 1/3, 1/3)^T$, 并令 $\alpha = 1/2$ 。按照 P 的定义, 从 A 出发时等概率到达 B, C , 从 B 出发时到达 C ; C 没有出链, 对应的列取为 v 。因此

$$P = \begin{pmatrix} 0 & 0 & 1/3 \\ 1/2 & 0 & 1/3 \\ 1/2 & 1 & 1/3 \end{pmatrix}.$$

若从均匀初值 $p^{(0)} = v$ 出发, 设稳定分布为 $(x, y, z)^T$, 代入式 (3.1) 可得

$$x = \frac{1}{6} + \frac{z}{6}, \quad y = \frac{1}{6} + \frac{x}{4} + \frac{z}{6}, \quad z = \frac{1}{6} + \frac{x}{4} + \frac{y}{2} + \frac{z}{6}.$$

解得 $p^* = (8/33, 10/33, 15/33)^T$ 。在这个例子中, 虽然 A 没有入链, 随机跳转仍能到达它, 因此它也有正分数。前面 HITS 的 authority 则只累加入链贡献, 所以 A 的 authority 为零。

不储存矩阵的实现

迭代公式 (3.1) 用矩阵乘法描述了一轮计算。然而现实中网页数可能达到数十亿, 存储完整的矩阵 P 显然不现实。事实上, 我们可以直接根据网页之间的链接计算下一轮分数, 只需保存邻接表和几个长度为 n 的向量。

具体而言, 记当前的概率向量为 p , 下一轮的结果为 q 。计算分为两步:

1. **先计算随机跳转和无出链网页的贡献。**首先对于所有网页, 随机跳转以概率 $1 - \alpha$ 发生, 目标网页按概率向量 v 选择, 因此跳转到网页 i 的概率是 $(1 - \alpha)v_i$ 。再将当前位于所有无出链网页的概率加起来, 记为 $s = \sum_{j:d_j=0} p_j$ 。这些网页在 P 中对应的列都是 v , 所以它们在 $\alpha P p$ 中跳转到网页 i 的概率合计为 $\alpha s v_i$ 。将这两部分相加, 就可以先令 $q_i = ((1 - \alpha) + \alpha s)v_i$ 。
2. **再沿已有链接累加贡献。**由于网页 j 有 d_j 条出链, 且当前位于它的概率为 p_j , 故以概率 α 沿链接浏览时, 对每条链接 $j \rightarrow i$, 只需将 $\frac{\alpha p_j}{d_j}$ 加到 q_i 上。遍历完所有链接后, 两种浏览方式带来的概率都已计入, q 就是下一轮的结果。

算法 3.4 PageRank 迭代算法

输入: $n \geq 1$ 的图, 概率向量 v , $0 < \alpha < 1$, $\varepsilon > 0$, 轮数上限 $K \geq 1$

```

1: 预计算每页出度  $d_j$ ;  $p \leftarrow v$ 
2: for  $t = 1, \dots, K$  do
3:    $s \leftarrow \sum_{j:d_j=0} p_j$ ;  $q \leftarrow ((1 - \alpha) + \alpha s)v$ 
4:   for 每条边  $j \rightarrow i$  do
5:      $q_i \leftarrow q_i + \alpha p_j / d_j$ 
6:   end for
7:    $\Delta \leftarrow \sum_i |q_i - p_i|$ ;  $p \leftarrow q$ 
8:   if  $\Delta \leq \varepsilon(1 - \alpha)/\alpha$  then
9:     return ( $p$ , 达到误差要求)
10:  end if
11: end for
```

12: **return** (p , 达到轮数上限, 误差条件尚未满足)

算法每轮需要遍历顶点和边, 时间为 $O(n + m)$, 故 K 轮共需 $O(K(n + m))$ 时间。图和工作向量共占 $O(n + m)$ 空间。由误差界 $2\alpha^t \leq \varepsilon$ 可知, 当 $0 < \varepsilon < 2$ 时, 取 $t \geq \left\lceil \frac{\log(2/\varepsilon)}{-\log \alpha} \right\rceil$ 轮足以使理论误差达到要求。

最后, 下表总结了 HITS 和 PageRank 的主要区别:

	HITS	PageRank
分数	authority 与 hub 两种	一个概率分数
贡献规则	hub 对每个链接目标贡献完整分数	按出度分配跳转概率
查询关系	通常对查询扩展后的子图计算	固定图和 v 后可预先计算
唯一性	最大特征值单重时, 全正初值得到相同极限	$0 < \alpha < 1$ 时唯一
无出链网页	hub 为零, 有入链时 authority 可非零	按 v 重新分配其跳转概率

如果希望评分偏向某个主题, 可以让跳转分布 v 给这一主题的网页更高概率, 这就是个性化 PageRank 的一种做法。

总结而言, 在搜索引擎中, 可以先用倒排索引找到候选网页, 再综合查询词的匹配情况、词项之间的距离和网页评分进行排序, 并通过评测选择各项因素的组合方式。

参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第三讲和提供的补充材料。
2. Manning、Raghavan、Schütze, [Introduction to Information Retrieval](#), Cambridge University Press, 2008, 第 1–5、8、21 章。
3. Kleinberg, [Authoritative Sources in a Hyperlinked Environment](#), JACM 46(5), 1999, 604–632。HITS 的算法及谱分析参照此文。
4. Page、Brin、Motwani、Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford 技术报告, 1999。